

Enabling Agent-Based Design

August 2024

Top Down

- Epics - Desired Outcomes

Bottom Up

- Epics - Delivered Outcomes

Overview

- **Key Challenge: We need a way to align Dev Teams (in Jira) with Customer needs**
 - How do we capture what customers want? (Use Case)
 - How do we define an “intent”? (is this the same as a scenario?)
 - How do we define success criteria to guide Testing?
 - How do we create visibility (end-2-end) between Strategy (desired value) & Execution (delivered value)?
- **Key Challenge: Need to improve Reuse of Models and Agents**
 - What naming conventions should we use for Models and Agents? (Search?)
 - How do we align Dev Teams on a taxonomy so we can organize Models & Agents?
 - What is the right level of granularity for a Model? (Intent 1:1 Model)
 - What about granularity for Agents? (macro vs micro)
- **Key Challenge: How do we introduce event-driven architecture for Agents?**
 - What is the intersection between Agents and Services?
 - What is the interaction between Agents and Services?
 - How do we govern changes across the landscape of Agents and services?
 - How do we scale build and run activities while reducing manual intervention?
 - How do we measure quality, compliance, and optimize productivity?

Leveraging Jira:

**Top Down (Customer Needs)
with**

Bottom Up (Agents & Solutions)

Enabling Value Chains with Agentic Design

Today, we always think about our technology in terms of solutions not necessarily services within a larger Value Stream (internally) and Value Chains (externally).

As we move forward, regulating bodies need the ability to trace & understand how AI models are consumed and reused.

The additional responsibilities as a Developer vs Deployer also come into play based on the type changes we make to an output along the way. This can get complicated as we step through the chain-of-delivery / custody.

Here's a simple outline that we may want to discuss further as a consistent framework from which to measure value | quality | trust | compliance | impact.

Value Chain

Value Stream(s)

Service Chain(s)

Process(s)

Input*

Processing*

Decision Logic

Discrete Activities

Output*

Measurement**

Measurement

Measurement

Measurement

Measurement

Controls

*Prevent

**Detect

Framing Design Time Services in Jira

Program Epic - L1 + L2 Service

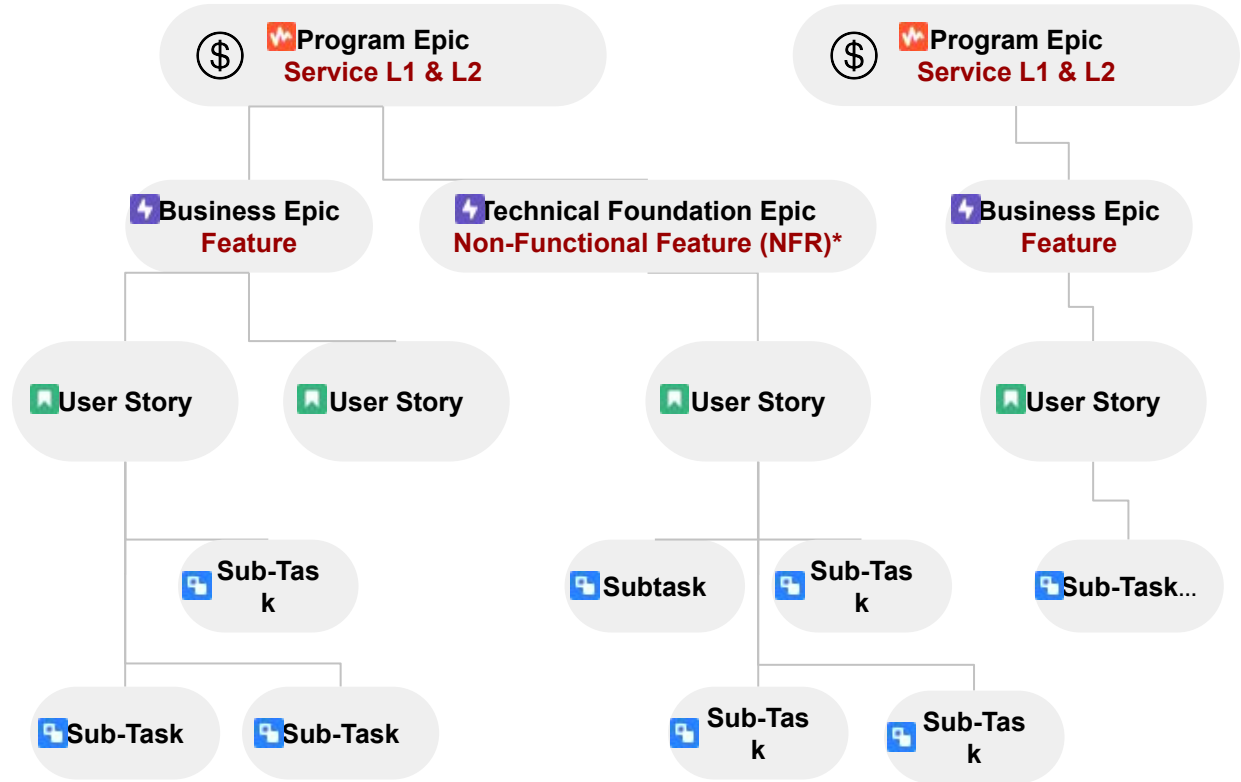
- Epic - Functional Feature
 - User Story (Use Case + Scenarios = Intents)
 - Sub-Task
 - Sub-Task...
- Epic - Non-Functional Feature*
 - User Story (Micro Service, Micro Agent)
 - Sub-Task
 - Sub-Task...

****Requirement:** all work items must map to the program hierarchy in Jira (to ensure funding)

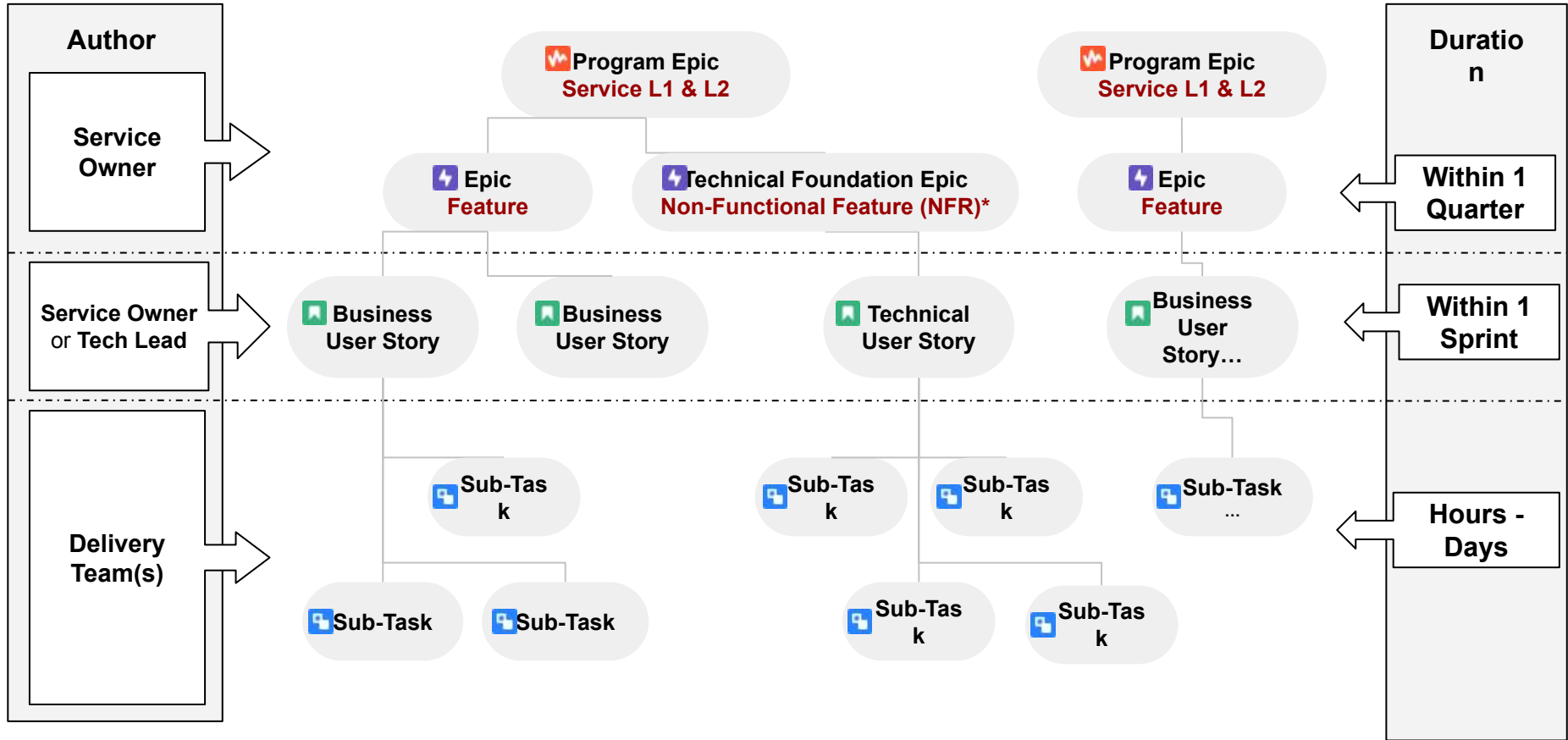
***Non-Functional Requirement (NFR)** is a requirement that does not relate to functionality, but to attributes such as reliability, efficiency, usability, maintainability and portability

Service Hierarchy in Jira

- Goal is to **link & label** all our work in Jira
- ***Technical Foundation / Non-Functional Requirement (NFR)** does not relate to functionality, but to attributes such as
 - Reliability
 - Efficiency
 - Usability
 - Maintainability
 - Portability
- The breakdown of technical work is captured in **Subtasks**
 - “How to” implementation steps



Authorship & Duration



Target State: Service Decomposition & Measurement

Value Chain

- Value Stream(s)
 - Service Chain(s)
 - Process(s)
 - Input*
 - Processing*
 - *Decision Logic*
 - *Discrete Activities*
 - Output*
 - Measurement**
 - Measurement
 - Measurement
 - Measurement
- Measurement

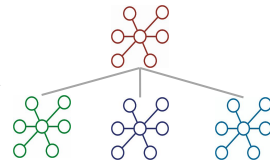
* Proactive

** Reactive

Measures need to be contextually relevant within each chain segment

Part of the challenge with aggregate measures and insights is balancing correlation with causation

Domain Specific KGs
(Knowledge Fabric)



Principles & Standards

Design Principles

Principle	Micro Service	Micro Agent
Bounded Context	Represents a single piece of business functionality.	Can play a single or multiple roles in a system.
Granularity	Should be small enough to ensure maintainability and extensibility.	Size/complexity is not an issue and often depends on the target domain.
Isolated State	Minimize sharing of state information across services.	State is local and private to an agent to support autonomy
Elasticity	The application is designed to allow addition and removal of required resources at runtime.	The ability to add/remove agents at run- time is a central feature of MAS.
Adaptability	Autonomous failure handling and ability to support scenario specific context	Autonomous Decisions Making, Continuous Learning, and Dynamic Tailoring
Loose Coupling	Orchestration and Choreography are not hard-wired (pre-determined) rather they are loosely coupled to support emergent behavior	Agents are autonomous loosely-coupled problem solvers.

Design Standards

- Partitioning by functional domain
- Agent composition introduces a micro-agent pattern where we can associate micro agents with micro services
- Each micro agent is tailored to it's complimentary micro service
- NL / Prompt interactions require durable yet elastic orchestration of micro agents working with micro services
 - Dynamic choreography
- Need for micro agents to clarify identity of Personas, the Jobs they are trying to get done, and their Desired Outcomes
 - Dynamic tailoring
- Orchestration defines directional flow (single thread and parallel)
 - Patterns will emerge over time
 - Flow within and across domains
- Choreography defines the interaction (interoperability) between agents and services
 - Patterns will emerge over time
 - Collaboration across domains is highly complex
- Context is the key to guide precision tailoring, orchestration, and choreography

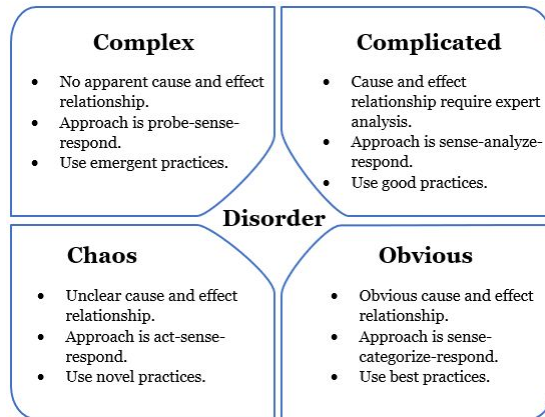
Agent Modeling

Domain Driven Design for Services

- Emergent behavior
- Augmented decision making
- Scenario driven adaptation
- Fine-grained services (scenario specific)

Intelligence Driven Design for Agents

- Continuous Learning
- Diagnostics Pattern recognition
- Problem classification ([cynefin](#))
- Augmented assistant

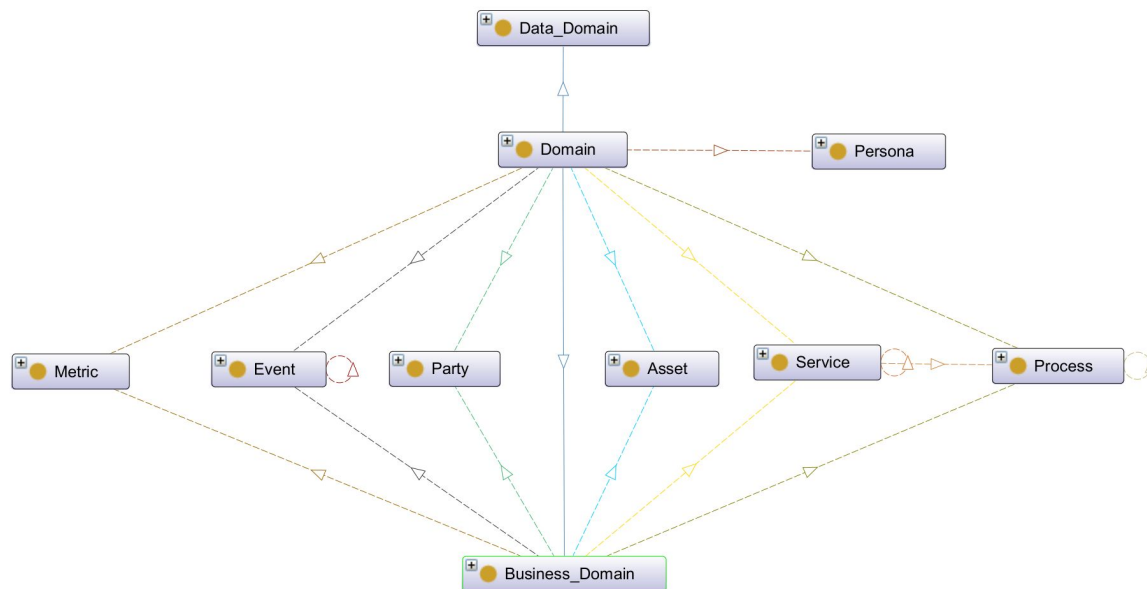


Breakdown for Services & Agents

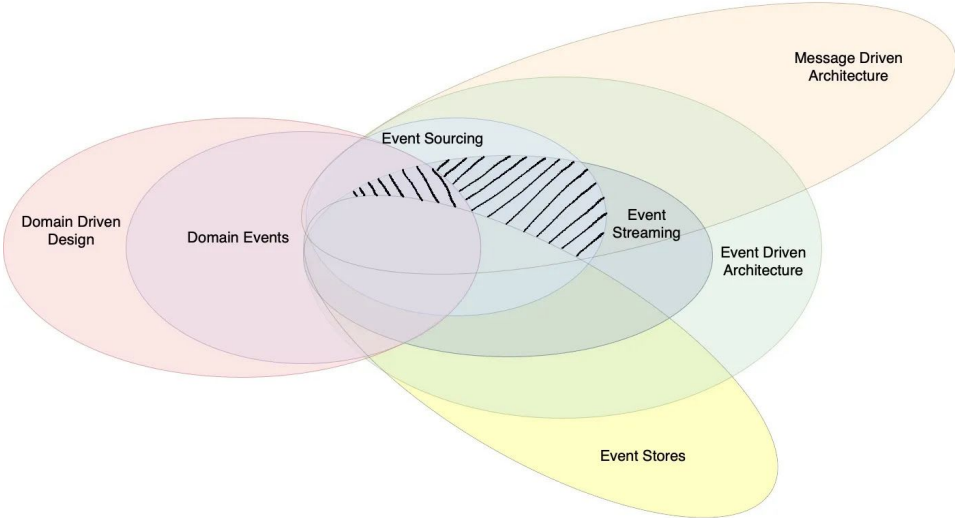
What is a Domain?

Domain Reference Model

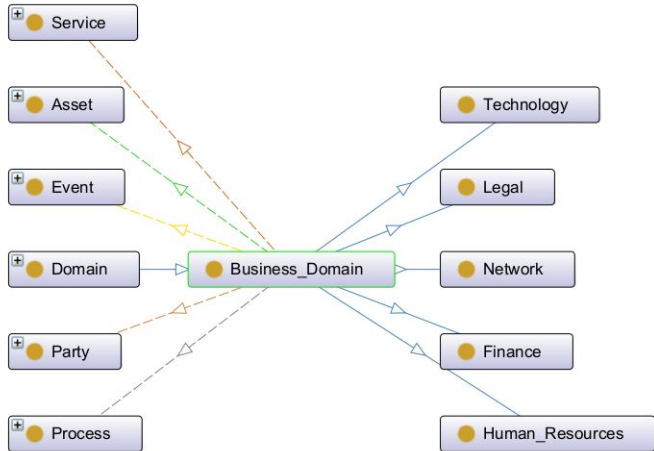
- Comprised of common set of classes
- Those classes are inherited by each Business Domain
- Services are a key part of a Domain. Each Domain has its own unique services.
- Each Domain may consume Services from other Domains



How do Events fit into Domain Driven Design?

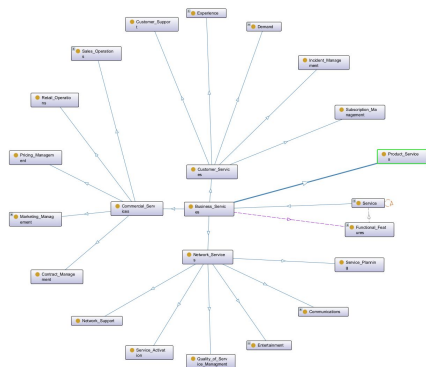


Events one of the components used to model Domain Context

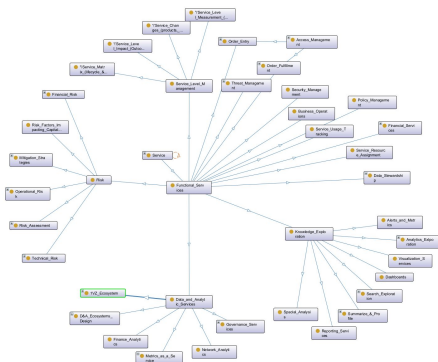


Domain Driven Design

- Commercial Services
- Customer Services
- Network Services
- Product Services
- “Expand Taxonomy WIP”



- Business Operations
- Data & Analytic Services
- Financial Services
- Human Resources
- Supply Chain
- “Expand Taxonomy WIP”



****Partial Listing ****

- Technical

- Data Services

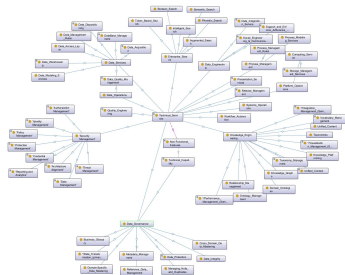
- Auditing
 - Instrumentation
 - Data Extraction
 - Data Transformation
 - Data Quality
 - Data Lineage
 - Data Loading

- Media Services

- Analytic Services

- Integration Services

- Human
 - Virtual
 - Physical



“Expand Taxonomy WIP”

- Technical (cont.)

- Operational Services

- Monitoring
 - Measuring
 - Anomaly Detection
 - Causal Analysis
 - Issue Management
 - Problem Resolution
 - Continuity Management
 - Escalation
 - Restoration
 - Notification

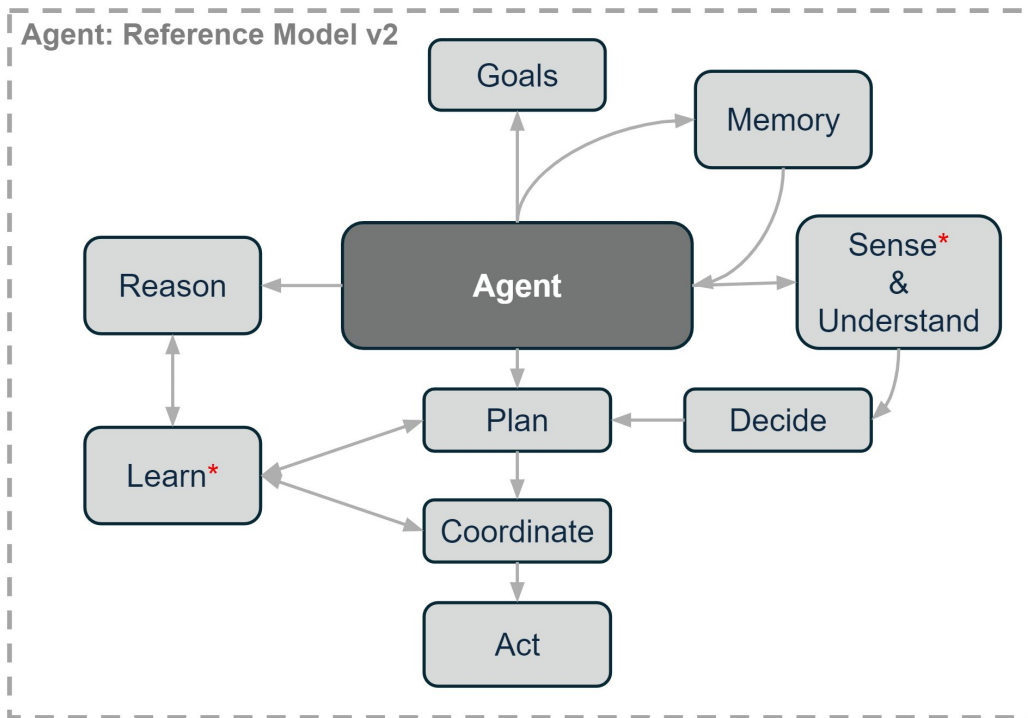
- Presentation Services

- UI / UX
 - Orchestration (Screenflow)
 - Visualization
 - Navigation
 - Voice
 - Avatars
 - Prompt Engineering

Target State Proposal...

Level 1 Micro Agents:

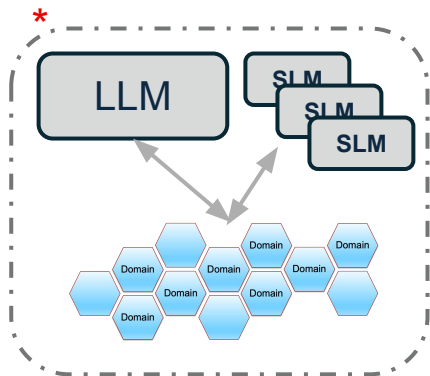
- Sensing
- Reasoning
- Perception
- Inference
- Projection
- Recognition
- Understanding
- Contextualization
- Simulation
- Consultation
- Decision Support
- Identification
- Provisioning
- Protection
- Optimization



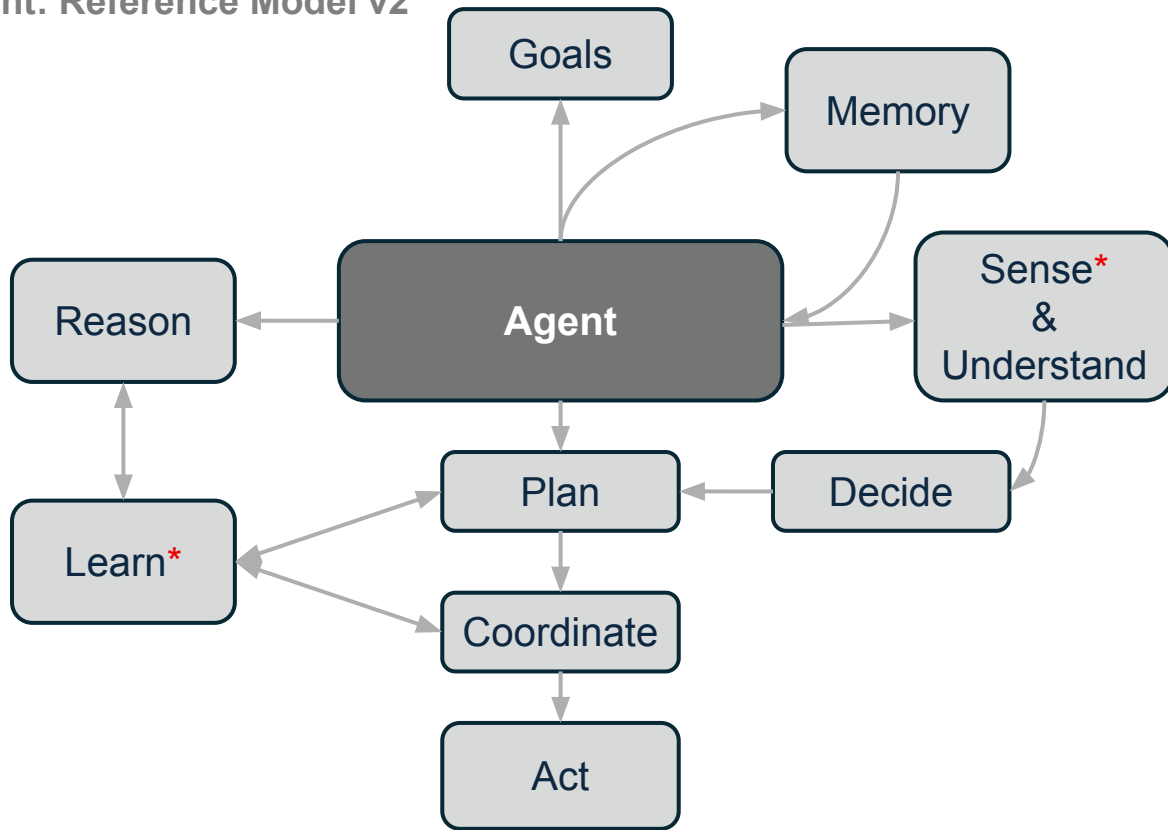
Emergent Intelligence via Agents (Target State)

Convergence of 4 separate disciplines:

- BI (analytic)
- AI (probabilistic)
- SI (deterministic)
- DI (decisions)



Agent: Reference Model v2



Use Case 1:M Solution(s)

Contextualizing a Use Case starts with where the UC is being applied

- Service
 - Process
 - Activity

Microservices, Logic, Events, AI (ML, GenAI, Agent), BI (Metric, Analytic, Agent) are all **components**.

Components need to be tied to solutions which support a Use Case

